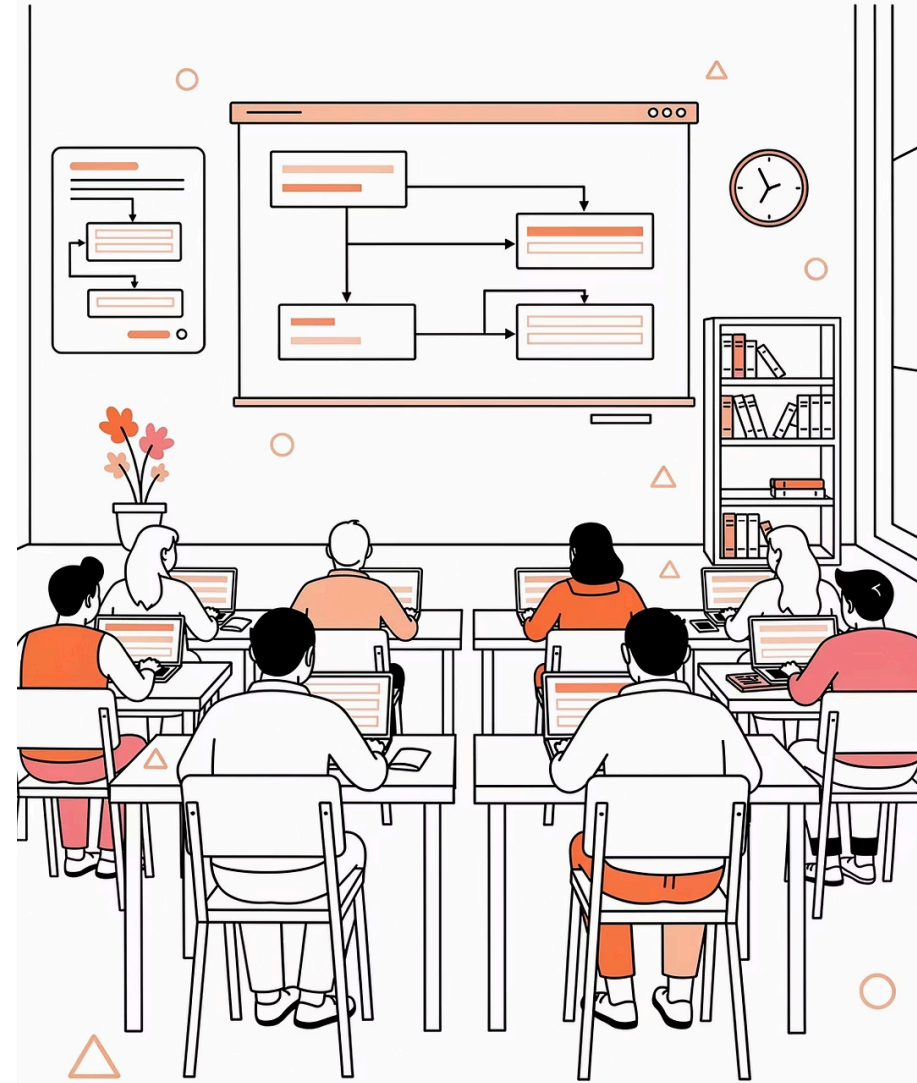


Fundamentos de Modelagem de Dados

BUSINESS INTELLIGENCE – AULA 2



Objetivos da Aula

Ao final desta aula, você estará preparado para compreender e aplicar os conceitos essenciais que fundamentam a modelagem de dados em ambientes corporativos. Estes conhecimentos são a base para qualquer profissional que deseja trabalhar com Business Intelligence e Analytics.

Conceitos Fundamentais

Compreender o que são bancos de dados, SGBDs e como os dados são organizados em estruturas relacionais.

Modelo Relacional

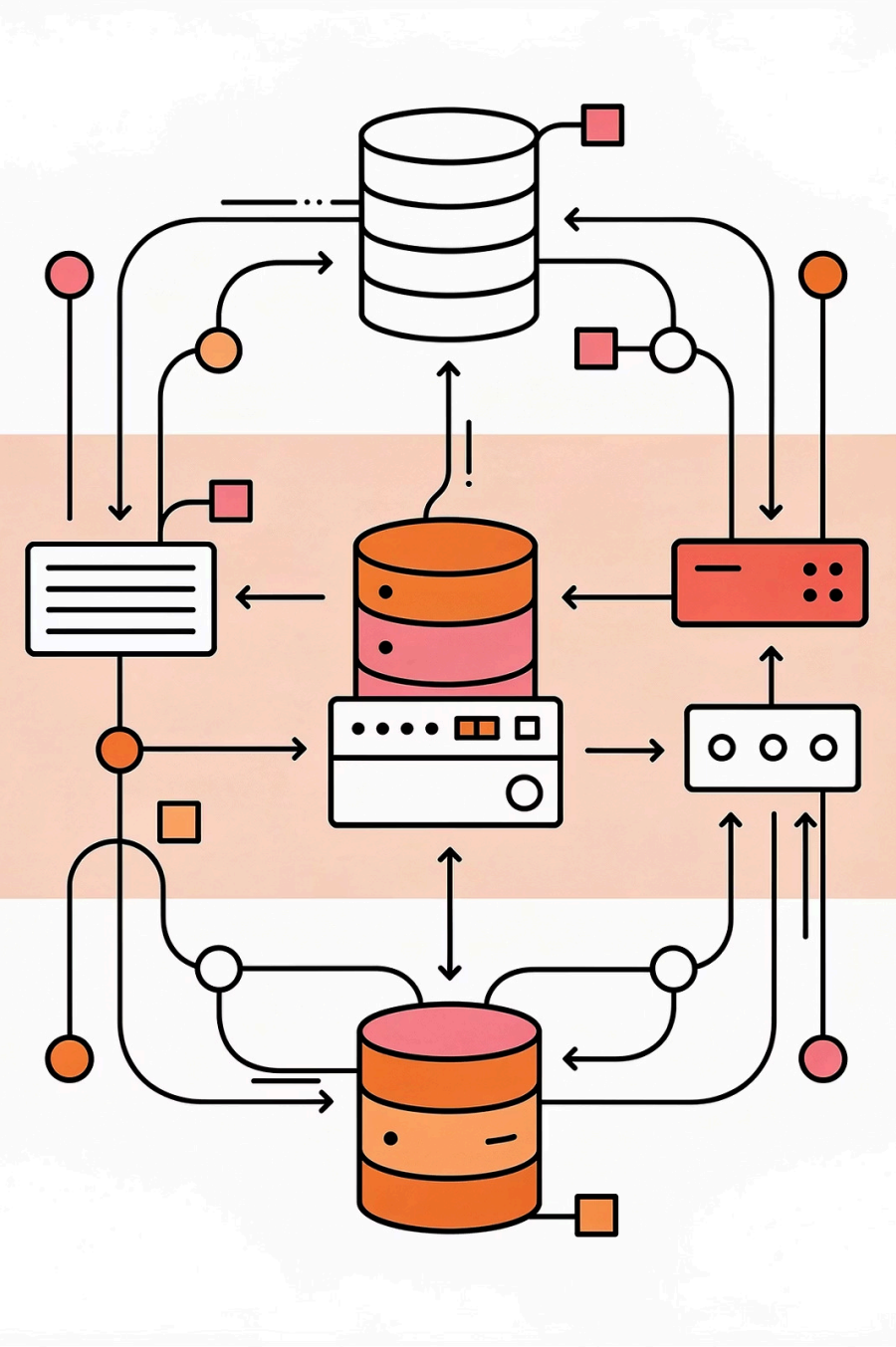
Entender a teoria por trás do modelo relacional, incluindo tabelas, chaves e relacionamentos entre entidades.

Normalização

Aplicar os princípios de normalização para eliminar redundâncias e garantir a integridade dos dados.

Limitações Analíticas

Identificar os problemas da modelagem tradicional quando aplicada em contextos de análise e tomada de decisão.



 PARTE 1

Conceitos de Banco de Dados

O que é um Banco de Dados?

Definição

Um banco de dados é um conjunto organizado de dados relacionados, armazenados de forma estruturada para facilitar consulta, atualização e análise eficiente.

Por que Bancos de Dados Existem?

- **Organizar informações:** Estruturar dados de forma lógica e acessível
- **Garantir integridade:** Assegurar que os dados sejam precisos e consistentes
- **Permitir recuperação eficiente:** Viabilizar consultas rápidas e confiáveis
- **Controlar acesso:** Gerenciar quem pode ver ou modificar informações

Sem bancos de **dados organizados**, empresas enfrentariam **duplicações**, **inconsistências** e **perda de informações** críticas para o negócio.

O que é um SGBD?

SGBD significa **Sistema Gerenciador de Banco de Dados**. É o software responsável por intermediar a comunicação entre os usuários e os dados armazenados, oferecendo funcionalidades essenciais para gestão e segurança.



MySQL

Sistema open-source popular, ideal para aplicações web e pequenas empresas.



PostgreSQL

Banco robusto e avançado, com suporte para dados complexos e análises.



SQL Server

Solução empresarial da Microsoft, integrada com ecossistema Windows.



Oracle

Líder em grandes corporações, oferecendo alta performance e escalabilidade.

Armazenar Dados

Persistir informações de forma organizada e recuperável

Controlar Acesso

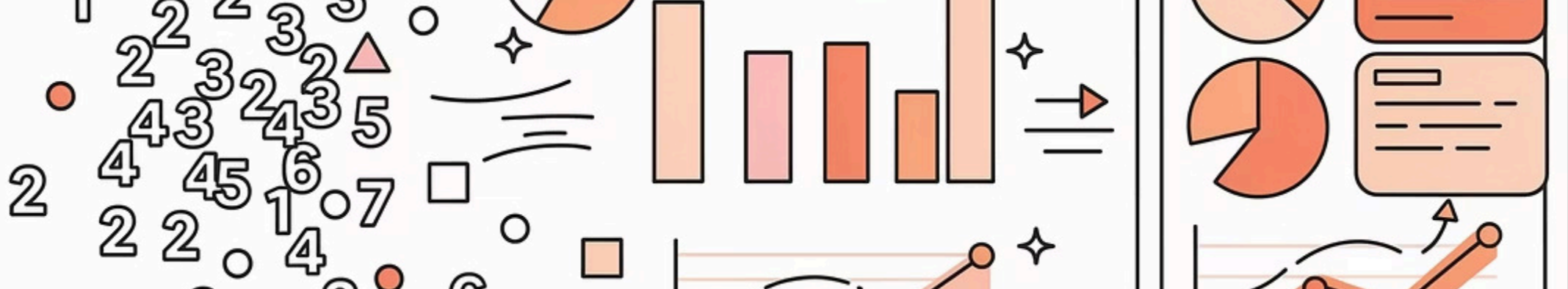
Definir permissões e níveis de autorização

Garantir Segurança

Proteger dados contra acessos não autorizados

Gerenciar Transações

Assegurar consistência em operações simultâneas



Dados vs Informação

Dado

É o **registro bruto**, sem contexto ou significado aparente. Representa apenas um valor isolado que, por si só, não oferece insights.

📌 **Exemplo:** O número 1500 sozinho não comunica nada relevante.

Informação

É o **dado com contexto**, processado e interpretado de forma que gera significado e valor para tomada de decisões.

📌 **Exemplo:** 1500 vendas realizadas em janeiro de 2024 representa uma informação acionável.

Business Intelligence transforma **dados** em **informação** e informação em **decisão estratégica**.

Estrutura Básica de um Banco Relacional

Um banco de dados relacional é composto por elementos fundamentais que trabalham juntos para **organizar** e **relacionar informações** de forma eficiente. Compreender esses componentes é essencial para modelar dados corretamente.



Tabelas

Estruturas que armazenam dados sobre uma entidade específica, como clientes ou produtos.



Linhas (Registros)

Cada linha representa uma instância única da entidade, como um cliente específico.



Colunas (Atributos)

Características ou propriedades da entidade, como nome, email ou data de nascimento.



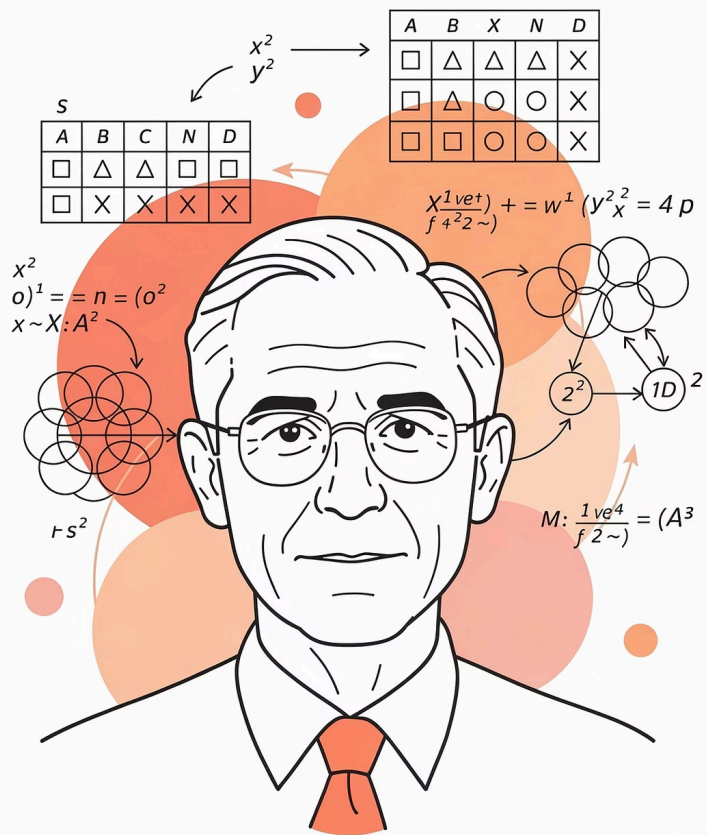
Chaves Primárias

Identificadores únicos que distinguem cada registro dentro de uma tabela.



Chaves Estrangeiras

Referências que conectam registros entre diferentes tabelas, criando relacionamentos.



Edgar F. Codd

 PARTE 2

Modelagem Relacional

O Modelo Relacional

O modelo relacional revolucionou a forma como armazenamos e gerenciamos dados. Criado por **Edgar F. Codd em 1970**, este modelo estabeleceu os fundamentos teóricos que ainda utilizamos hoje em sistemas de bancos de dados modernos.

O modelo organiza dados em **tabelas relacionadas entre si**, onde cada tabela representa uma entidade do mundo real. As relações entre tabelas são estabelecidas através de chaves, permitindo consultas complexas e mantendo a integridade dos dados.

Esta abordagem eliminou muitas das limitações dos modelos anteriores, como os sistemas hierárquicos e em rede, oferecendo maior flexibilidade e simplicidade.

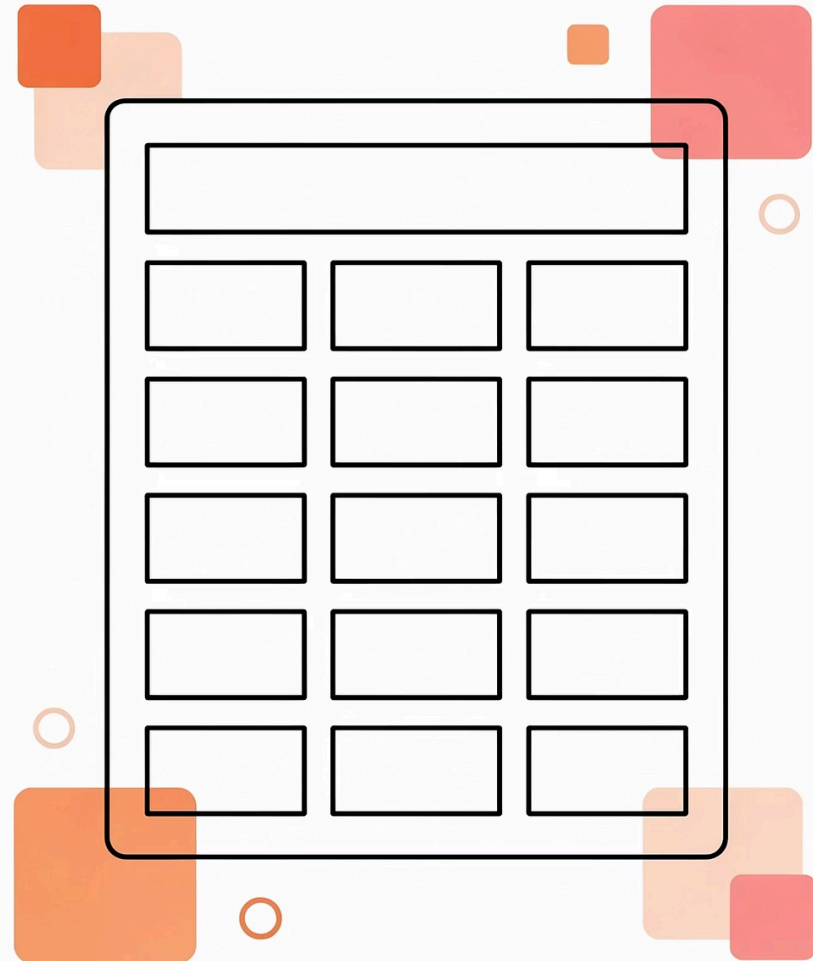
Tabela: O Elemento Central

Uma tabela é um **conjunto de registros sobre uma entidade específica**. Ela organiza informações de forma estruturada, onde cada coluna representa um atributo e cada linha representa uma instância única.

Exemplo: Tabela CLIENTE

- **id_cliente:** Identificador único (chave primária)
- **nome:** Nome completo do cliente
- **email:** Endereço de email para contato
- **cidade:** Localização geográfica
- **data_cadastro:** Quando o cliente foi registrado no sistema

Cada cliente cadastrado no sistema ocupa uma linha nesta tabela, com seus dados específicos preenchendo as respectivas colunas.



Chave Primária (Primary Key)

A chave primária é o elemento que **identifica de forma única cada registro** dentro de uma tabela. Ela é fundamental para garantir que não existam duplicações e para permitir relacionamentos confiáveis entre tabelas.



Não Se Repete

Cada valor de chave primária deve ser único na tabela. Dois registros nunca podem ter o mesmo valor.



Não Pode Ser Nula

Todo registro deve ter um valor válido na chave primária. Valores nulos não são permitidos.



Deve Ser Estável

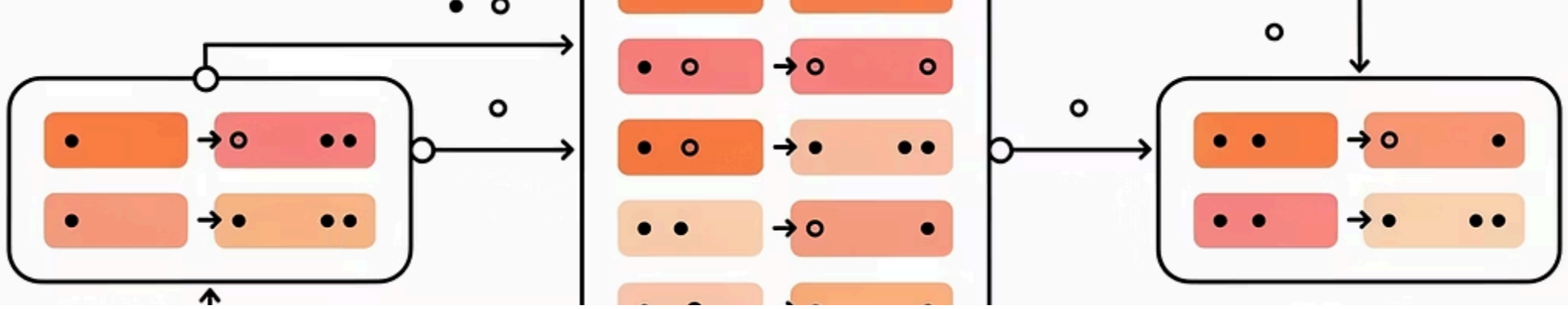
Uma vez definida, a chave primária não deve mudar ao longo do tempo para manter a integridade referencial.



Exemplo: id_cliente = 1001

Este identificador único permanece associado ao mesmo cliente durante toda sua existência no sistema.

Chaves primárias são frequentemente **números inteiros sequenciais** ou **identificadores únicos universais** (UUID), garantindo unicidade e performance nas operações de busca.



Chave Estrangeira (Foreign Key)

A chave estrangeira é o mecanismo que **relaciona tabelas** entre si, criando conexões lógicas entre diferentes entidades. Ela referencia a chave primária de outra tabela, estabelecendo um vínculo de integridade referencial.

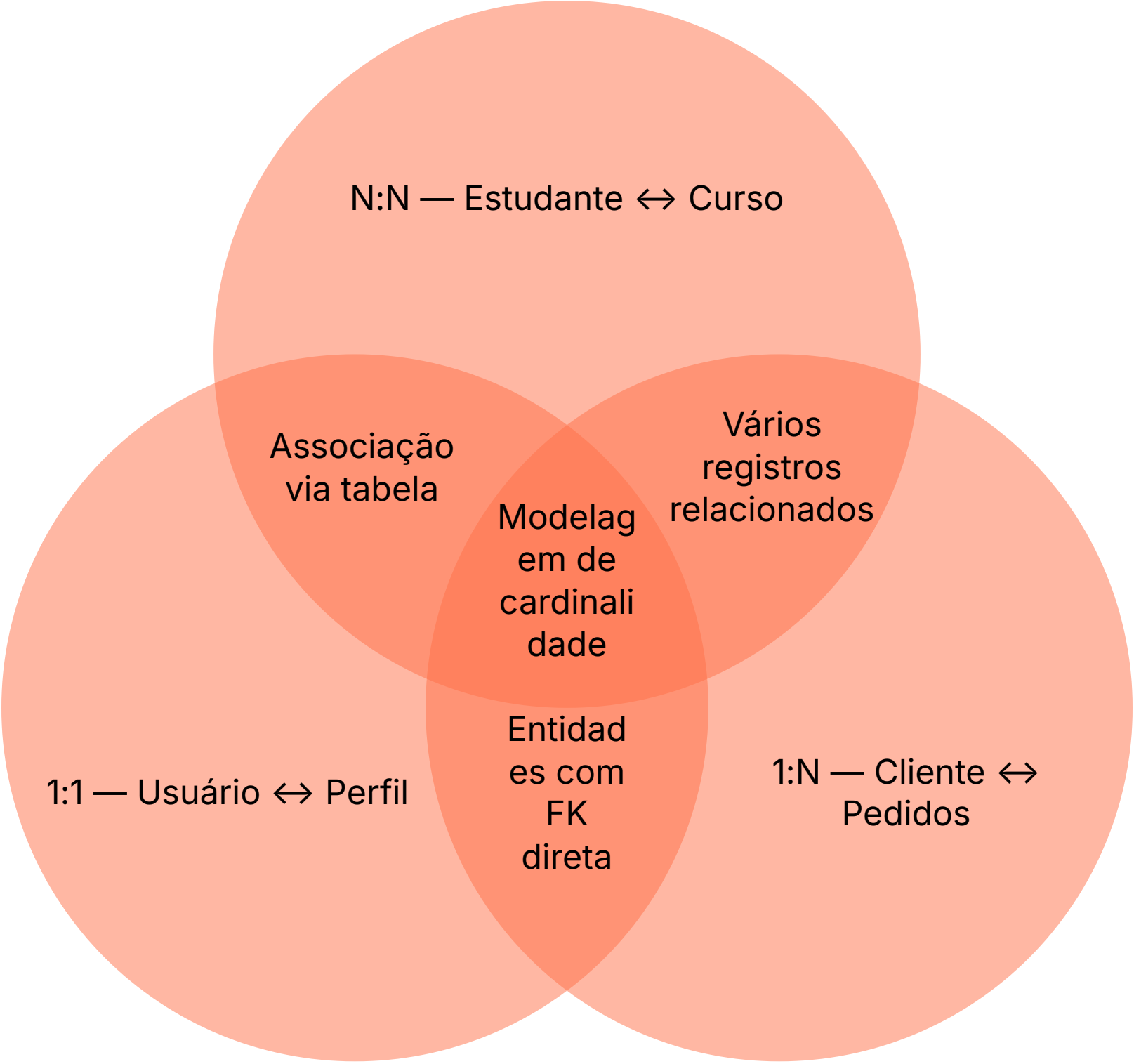
Exemplo: Tabela PEDIDO

- **id_pedido:** Chave primária da tabela Pedido
- **id_cliente (FK):** Chave estrangeira que referencia Cliente
- **data:** Data de realização do pedido
- **valor:** Valor total do pedido
- **status:** Situação atual do pedido

Através do campo `id_cliente`, podemos identificar qual cliente realizou cada pedido, mantendo a integridade dos dados e evitando duplicação de informações do cliente em cada pedido.

Tipos de Relacionamentos

Os relacionamentos entre tabelas podem assumir diferentes **cardinalidades**, dependendo de como as entidades se conectam no mundo real. Compreender esses padrões é essencial para uma modelagem correta.



A modelagem correta dos relacionamentos evita redundância e inconsistência nos dados, garantindo que as regras de negócio sejam respeitadas pela estrutura do banco de dados.

1:1 (Um-para-Um)

Cada registro em uma tabela relaciona-se com apenas um registro em outra. Ex: Usuário → Perfil

1:N (Um-para-Muitos)

Um registro em uma tabela relaciona-se com vários em outra. Ex: Cliente → Pedidos

N:N (Muitos-para-Muitos)

Múltiplos registros de ambas as tabelas relacionam-se. Ex: Estudantes ↔ Cursos

Exemplo de Modelo Relacional

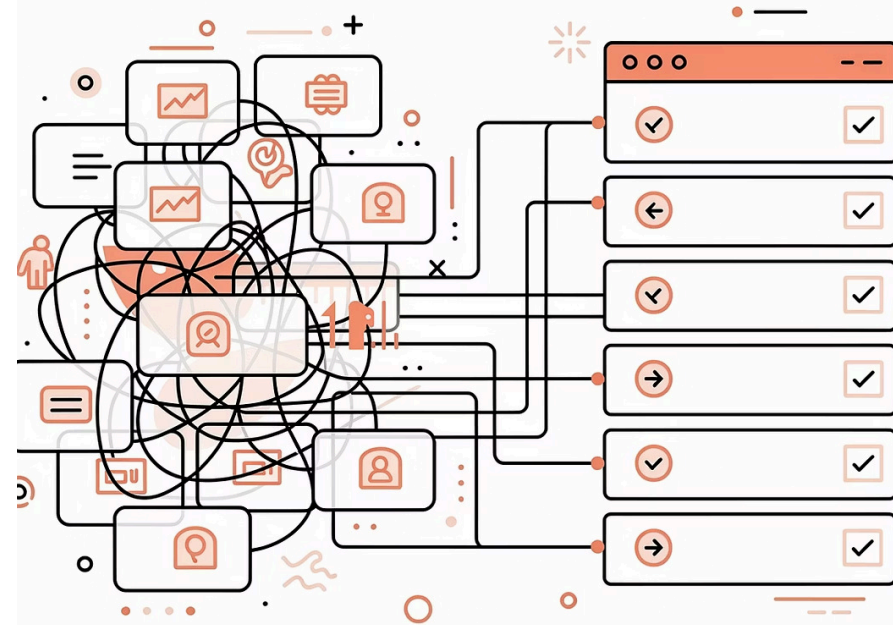
Vamos analisar um cenário típico de uma **loja virtual** para entender como as tabelas se relacionam em um sistema real.

Pergunta para reflexão: Este modelo serve bem para análise estratégica? Ou ele foi projetado principalmente para operações transacionais?

Note que este modelo é **excelente para processar vendas diárias**, mas pode se tornar complexo quando precisamos responder perguntas analíticas como "**qual categoria de produto gerou mais receita por região nos últimos 12 meses?**"

PARTE 3

Normalização de Dados



O que é Normalização?

Normalização é um **processo sistemático de organização dos dados** em um banco relacional, aplicando regras específicas para estruturar as tabelas de forma otimizada. É uma técnica fundamental para **garantir a qualidade e consistência das informações**.



Eliminar Redundância

Remover dados duplicados que ocupam espaço desnecessário e podem gerar inconsistências.



Evitar Anomalias

Prevenir problemas em operações de inserção, atualização e exclusão de dados.



Garantir Consistência

Assegurar que os dados permaneçam íntegros e corretos ao longo do tempo.

O processo de normalização segue formas normais progressivas (**1FN**, **2FN**, **3FN** e além), cada uma adicionando regras mais rigorosas para refinar a estrutura do banco de dados.

Problemas Sem Normalização

Quando um banco de dados não é adequadamente normalizado, diversos problemas graves podem surgir, comprometendo a **integridade** e a **confiabilidade** das informações armazenadas.

Consequências Críticas

Bancos não normalizados geram custos de manutenção, erros operacionais e perda de confiança nos dados.

Dados Duplicados

Informações repetidas ocupam espaço desnecessário e aumentam custos de armazenamento.

Atualizações Inconsistentes

Alterações podem não se propagar para todas as cópias, criando versões conflitantes.

Exclusões Problemáticas

Remover um registro pode causar perda não intencional de outras informações.

Inserções Limitadas

Impossibilidade de adicionar dados sem ter informações completas de outras entidades.

Primeira Forma Normal (1FN)

A Primeira Forma Normal (1FN) é o **primeiro nível de normalização** de um banco de dados relacional. Ela garante que a tabela esteja organizada de forma básica e estruturada, evitando ambiguidades e dados inconsistentes.

1

Regra Essencial da 1FN

Uma tabela está na 1FN quando **cada célula contém apenas um único valor indivisível** (atômico).

2

Propósito

Elimina a presença de grupos repetitivos e atributos multivalorados dentro de uma única célula, garantindo a integridade fundamental dos dados.

Três Critérios Fundamentais da 1FN

Para que uma tabela seja considerada na Primeira Forma Normal (1FN), ela deve aderir a três critérios essenciais que garantem a atomicidade e a unicidade dos dados em sua forma mais básica.

1. Valores Atômicos

Cada campo deve conter apenas um valor indivisível. Não são permitidas listas ou múltiplos itens na mesma célula.

✗ **Incorreto:** Telefone = (61)9999-9999, (61)9888-8888

✓ **Correto:** Uma linha para cada telefone.

2. Ausência de Grupos Repetitivos

Não pode haver colunas repetidas para o mesmo tipo de informação dentro da mesma tabela.

✗ **Incorreto:** Produto1 | Produto2 | Produto3

✓ **Correto:** Criar linhas adicionais ou tabelas separadas para cada produto.

3. Identificação Única (Chave Primária)

Cada registro da tabela deve ser identificado de forma única por uma chave primária.

Isso evita duplicidade de registros e garante a integridade referencial.

O **objetivo da 1FN** é organizar os dados para evitar redundância básica e preparar a base para as próximas formas normais (2FN e 3FN), garantindo uma estrutura de banco de dados mais eficiente e robusta.

Primeira Forma Normal (1FN)

Eliminando atributos multivalorados e garantindo atomicidade

Uma tabela está na 1FN quando:

- Não possui atributos multivalorados ou repetidos
- Todos os atributos são atômicos (não podem ser divididos)
- Possui uma chave primária que identifica cada linha de forma única

⚠️ Valores separados por vírgulas, listas ou conjuntos em uma única coluna violam a 1FN!

Exemplo: Tabela fora da 1FN

Cliente	Telefones
Ana Silva	99887766, 88776655

Transformação para 1FN

Cliente	Telefone
Ana Silva	99887766
Ana Silva	88776655

Exercício: Aplicando a 1FN

Identifique e corrija os problemas de 1FN na tabela abaixo

ID_Aluno	Nome	Disciplinas	Endereço
1	João Silva	Banco de Dados, Programação	Rua A, 123 - Centro - Florianópolis
2	Maria Santos	Redes, Sistemas Operacionais, IA	Av. B, 456 - Trindade - Florianópolis

Problemas:

- 1. O atributo "Disciplinas" contém valores múltiplos separados por vírgulas
- 2. O atributo "Endereço" não é atômico, podendo ser dividido em partes menores

Resolução do Exercício: Aplicando a 1FN

Problemas identificados na tabela original:

- O atributo "Disciplinas" contém valores múltiplos separados por vírgulas (viola a 1FN)
- O atributo "Endereço" não é atômico, podendo ser dividido em partes menores (viola a 1FN)

Solução para o problema das disciplinas:

Criar uma tabela separada para relacionar alunos e disciplinas:

Tabela Aluno:

ID_Aluno	Nome
1	João Silva
2	Maria Santos

Tabela Disciplina:

ID_Disciplina	Nome_Disciplina
1	Banco de Dados
2	Programação
3	Redes
4	Sistemas Operacionais
5	IA

Tabela Aluno_Disciplina:

ID_Aluno	ID_Disciplina
1	1
1	2
2	3
2	4
2	5

Solução para o problema do endereço:

Decompor o endereço em atributos atômicos:

Tabela Aluno (versão final em 1FN):

ID_Aluno	Nome	Logradouro	Numero	Bairro	Cidade
1	João Silva	Rua A	123	Centro	Florianópolis
2	Maria Santos	Av. B	456	Trindade	Florianópolis

Agora todas as tabelas estão na Primeira Forma Normal (1FN)!

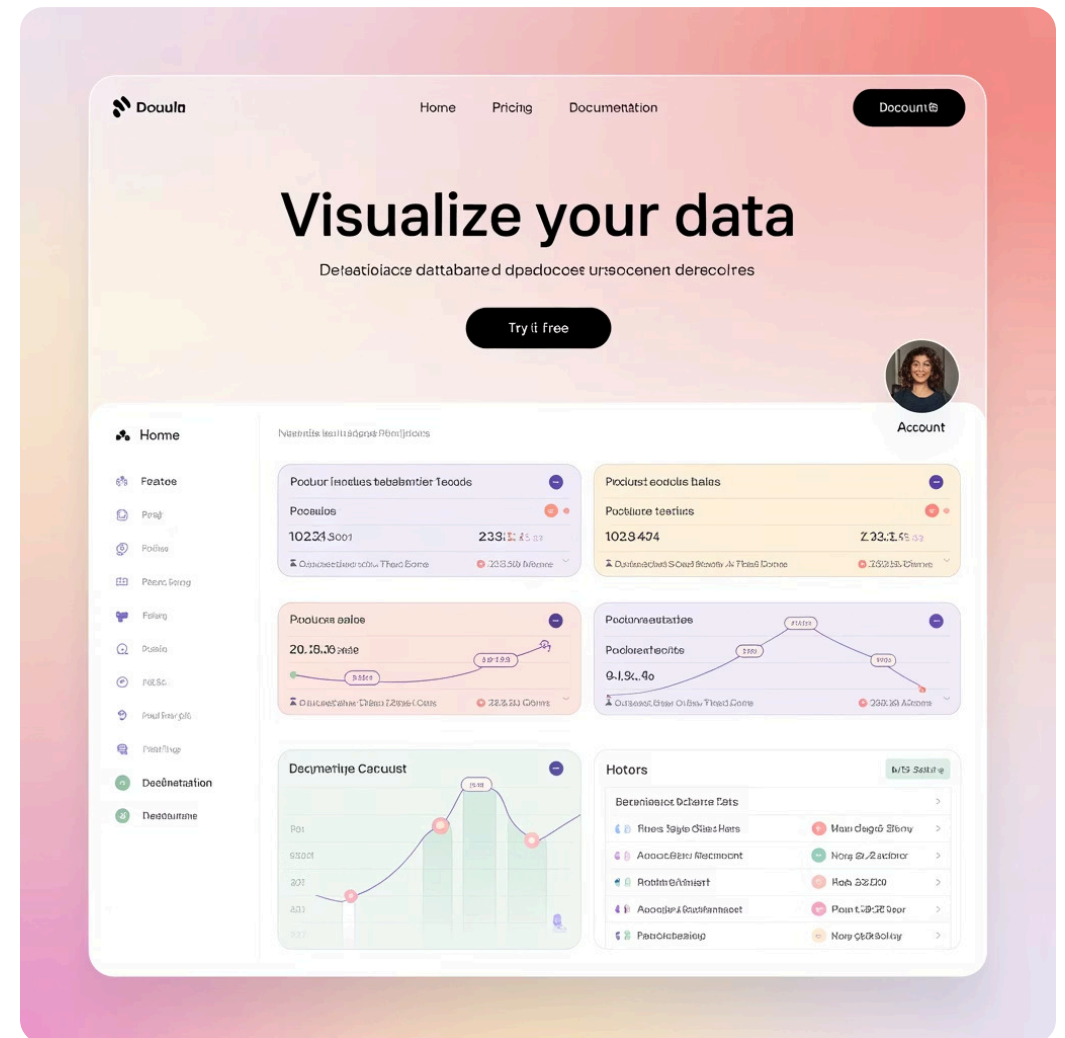
Segunda Forma Normal (2FN)

Eliminando dependências parciais da chave primária

Uma tabela está na 2FN quando:

- Já está na 1FN
- Todos os atributos não-chave dependem completamente da chave primária (e não apenas de parte dela)

A 2FN só é relevante quando temos uma chave primária composta. Se a chave primária for simples (um único atributo), a tabela em 1FN já está automaticamente em 2FN.



❗ Na prática, a 2FN busca identificar quando um atributo depende apenas de parte da chave primária composta, o que indica a necessidade de criar uma nova tabela.

Exemplo: Aplicando a 2FN

Tabela Matrícula fora da 2FN

ID_Aluno	ID_Disciplina	Nome_Aluno	Nome_Disciplina	Nota
1	101	João Silva	Banco de Dados	8.5
1	102	João Silva	Programação	7.0
2	101	Maria Santos	Banco de Dados	9.0

Problemas:

- A chave primária é composta: (ID_Aluno, ID_Disciplina)
- Nome_Aluno depende apenas de ID_Aluno (dependência parcial)
- Nome_Disciplina depende apenas de ID_Disciplina (dependência parcial)

Transformação para 2FN

Tabela Aluno

ID_Aluno	Nome_Aluno
1	João Silva
2	Maria Santos

Tabela Disciplina

ID_Disciplina	Nome_Disciplina
101	Banco de Dados
102	Programação

Tabela Matrícula (agora em 2FN)

ID_Aluno	ID_Disciplina	Nota
1	101	8.5
1	102	7.0
2	101	9.0

Exercício: Aplicando a 2FN

Identifique e corrija os problemas de 2FN na tabela abaixo

ID_Produto	ID_Pedido	Nome_Produto	Preço_Produto	Data_Pedido	Quantidade
101	1	Monitor LED	1200,00	10/03/2023	1
102	1	Teclado	150,00	10/03/2023	2
101	2	Monitor LED	1200,00	15/03/2023	3

Identifique:

1. A chave primária composta
2. As dependências parciais
3. Como dividir esta tabela para alcançar a 2FN

Resolução do Exercício: Aplicando a 2FN

Análise da tabela original:

Chave primária composta: (ID_Produto, ID_Pedido)

Dependências identificadas:

- Nome_Produto depende apenas de ID_Produto (dependência parcial)
- Preço_Produto depende apenas de ID_Produto (dependência parcial)
- Data_Pedido depende apenas de ID_Pedido (dependência parcial)
- Quantidade depende da chave primária completa (ID_Produto, ID_Pedido) ✓

Solução: Dividir em três tabelas para alcançar a 2FN

Tabela Produto:

ID_Produto	Nome_Produto	Preço_Produto
101	Monitor LED	1200,00
102	Teclado	150,00

Tabela Pedido:

ID_Pedido	Data_Pedido
1	10/03/2023
2	15/03/2023

Tabela Item_Pedido (relacionamento N:N):

ID_Produto	ID_Pedido	Quantidade
101	1	1
102	1	2
101	2	3

Agora todas as tabelas estão na Segunda Forma Normal (2FN)!

- Eliminamos as dependências parciais
- Cada atributo não-chave depende completamente da chave primária
- Reduzimos redundâncias e anomalias de atualização

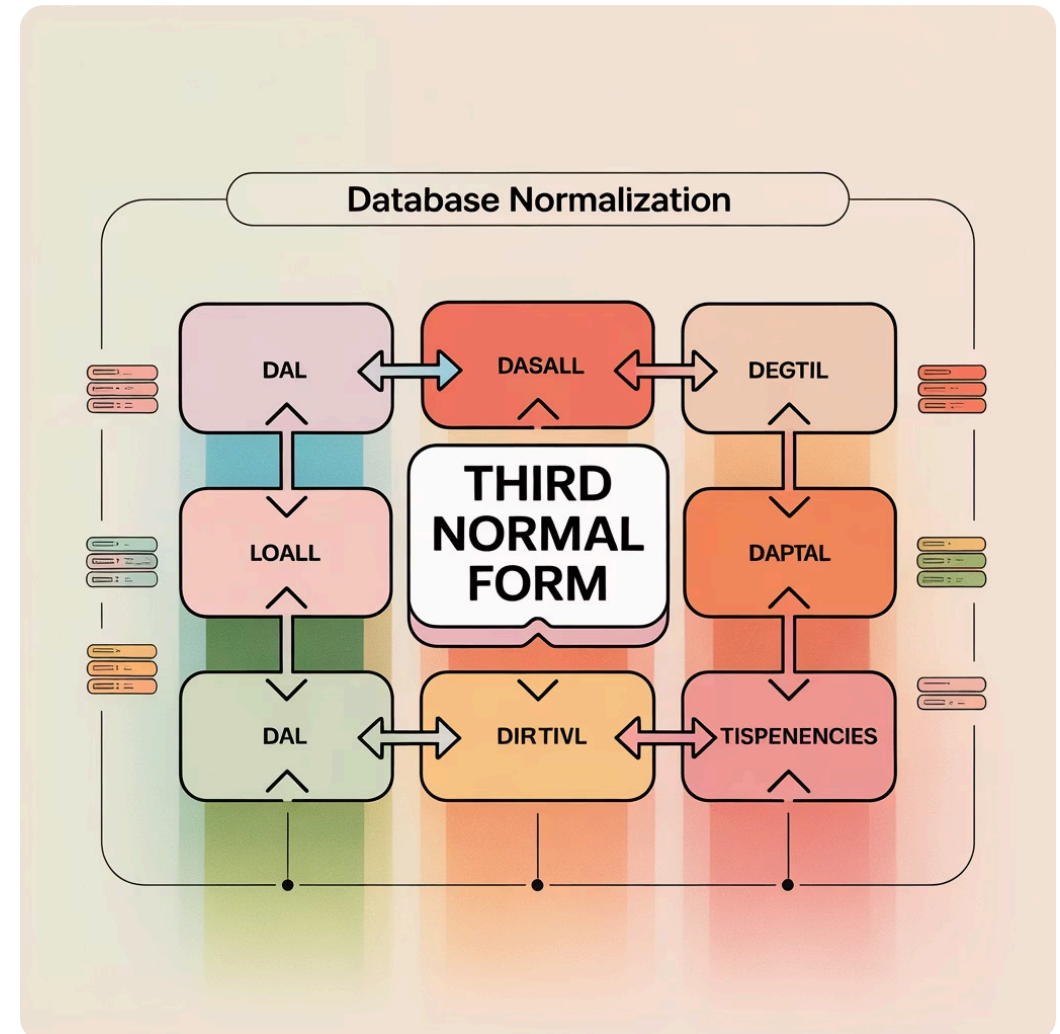
Terceira Forma Normal (3FN)

Eliminando dependências transitivas

Uma tabela está na 3FN quando:

- Já está na 2FN
- Não possui dependências transitivas (quando um atributo não-chave depende de outro atributo não-chave)

A 3FN busca eliminar casos onde um atributo depende de outro que não faz parte da chave primária, criando novas tabelas quando necessário.



- i** Na prática, a 3FN elimina redundâncias que surgem quando um atributo pode ser derivado de outro atributo não-chave, como no caso de códigos e suas descrições.

Exemplo: Aplicando a 3FN

Tabela Funcionário fora da 3FN

ID_Func	Nome	ID_Depto	Nome_Depto	Localização_Depto
1	Ana Silva	10	TI	Bloco A
2	Carlos Pereira	10	TI	Bloco A
3	Maria Santos	20	RH	Bloco B

Problema: Nome_Depto e Localização_Depto dependem de ID_Depto, não da chave primária ID_Func.

Esta é uma dependência transitiva: ID_Func → ID_Depto → Nome_Depto e Localização_Depto

Transformação para 3FN

Tabela Funcionário (agora em 3FN)

ID_Func	Nome	ID_Depto
1	Ana Silva	10
2	Carlos Pereira	10
3	Maria Santos	20

Tabela Departamento

ID_Depto	Nome_Depto	Localização_Depto
10	TI	Bloco A
20	RH	Bloco B

A transformação para 3FN eliminou a redundância de dados de departamento e evita anomalias de atualização (ex: alterar o nome de um departamento exigiria atualizar várias linhas na tabela original).

Resumo das Formas Normais

Primeira Forma Normal (1FN)

Elimina atributos multivalorados e garante atomicidade.

Cada atributo deve conter apenas um valor e ser indivisível.

Segunda Forma Normal (2FN)

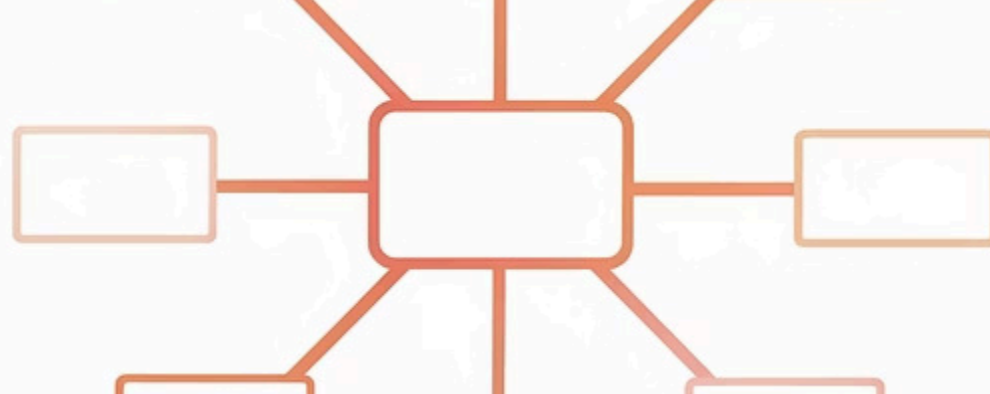
Elimina dependências parciais da chave primária.

Todos os atributos não-chave dependem completamente da chave primária.

Terceira Forma Normal (3FN)

Elimina dependências transitivas.

Nenhum atributo não-chave depende de outro atributo não-chave.



PARTE 4

Problemas da Modelagem Tradicional para BI

Modelagem Relacional é Ótima Para...

O modelo relacional normalizado foi projetado especificamente para **sistemas transacionais (OLTP - Online Transaction Processing)**. Neste contexto, ele oferece vantagens significativas que justificam sua ampla adoção.



Inserções Rápidas

Otimizado para adicionar novos registros rapidamente, essencial em operações como vendas e cadastros em tempo real.



Atualizações Constantes

Permite modificações frequentes de dados com garantia de consistência através de transações.



Integridade Operacional

Regras de validação e constraints garantem que apenas dados válidos sejam inseridos no sistema.



Controle de Concorrência

Gerencia múltiplos usuários acessando e modificando dados simultaneamente sem conflitos.

Mas ela tem Limitações para BI

Quando aplicamos a modelagem relacional normalizada em cenários de **análise e Business Intelligence**, surgem desafios significativos que impactam performance e usabilidade. O que funciona perfeitamente para transações diárias não é ideal para análises estratégicas.

Muitas Tabelas

A normalização fragmenta dados em dezenas de tabelas, dificultando a compreensão do modelo.

Muitas Junções (JOINS)

Consultas analíticas exigem combinar 5, 10 ou mais tabelas, aumentando a complexidade.

Consultas Lentas

O processamento de múltiplos JOINS consome tempo e recursos computacionais significativos.

Complexidade para Usuários

Analistas de negócio precisam entender estruturas técnicas complexas para extrair insights.

O Dilema OLTP vs OLAP

Sistemas transacionais priorizam velocidade de escrita e integridade. Sistemas analíticos priorizam velocidade de leitura e simplicidade de consulta.

São necessidades conflitantes que exigem abordagens de modelagem diferentes.

Exemplo de Problema Analítico

Vamos analisar uma pergunta de negócio aparentemente simples e observar a complexidade que ela gera em um modelo relacional normalizado.

Pergunta de Negócio: Qual o faturamento por cidade nos últimos 12 meses?

Consulta Necessária no Modelo Relacional:

```
SELECT
  c.cidade,
  SUM(ip.quantidade * ip.valor_unitario) as faturamento
FROM Cliente c
JOIN Pedido p ON c.id_cliente = p.id_cliente
JOIN ItemPedido ip ON p.id_pedido = ip.id_pedido
JOIN Produto prod ON ip.id_produto = prod.id_produto
WHERE p.data >= DATE_SUB(NOW(), INTERVAL 12 MONTH)
GROUP BY c.cidade
ORDER BY faturamento DESC
```

Esta consulta **requer 4 JOINS** para responder uma pergunta simples. Imagine consultas mais complexas envolvendo categorias de produtos, regiões, períodos comparativos e segmentos de clientes. A complexidade cresce exponencialmente.

❏ Em um modelo dimensional, esta mesma consulta seria muito mais simples e rápida, pois os dados já estariam pré-organizados para análise.